

FC7xxx DMA User Manual

Rev.0.4

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/11/22	Updated for MCAL V0.4.0

Table of Contents

Chapter 1 DMA Introduction	5
1.1 Requirement Tracing	5
1.2 Introduction to CDD	5
1.3 Hardware Summary	5
Chapter 2 Software Design	7
2.1 Rejected Requirements	7
2.2 File Structure	7
2.3 Macros	7
2.3.1 Macros in CDD_Dma.c	7
2.3.2 Macros in CDD_Dma.h	7
2.3.3 Macros in Dma_Types.h	11
2.4 Enums	11
2.4.1 Enumerations in Dma_Types.h	11
2.5 Typedef	16
2.5.1 Typedefs in Dma_Types.h	16
2.6 Structures	16
2.6.1 Dma_ChannelConfigType	16
2.6.2 Dma_ChannelType	17
2.6.3 Dma_CoreSpecificChConfigType	18
2.6.4 Dma_CoreSpecificHwUnitConfigType	18
2.6.5 Dma_ConfigType	19
2.7 API Functions	19
2.7.1 Functions in Cdd_Dma.c	19
2.8 WAPI Sequence Diagram	30
2.8.1 DMA Initialization and Requested by Software	30
2.8.2 DMA Initialization and Requested by Hardware	31
Chapter 3 Tresos Configuration Items	32
3.1 Container Inclusion Relation	32
3.2 Containers and Variables	32
3.2.1 IMPLEMENTATION_CONFIG_VARIANT	32
3.2.2 DMAGeneral	32
3.2.3 DMAConfigSet	34
3.2.4 DmaChannelConfig	37
3.2.5 CommonPublishedInformation	40
Chapter 4 Configuration Guides	43

4.1	Configuration Item Constraint	43
4.2	DMA Usage Common Steps	44
4.3	DMA Configuration Demo	45
4.3.1	Hardware Trigger	45
4.3.2	Software Trigger	45

Chapter 1 DMA Introduction

1.1 Requirement Tracing

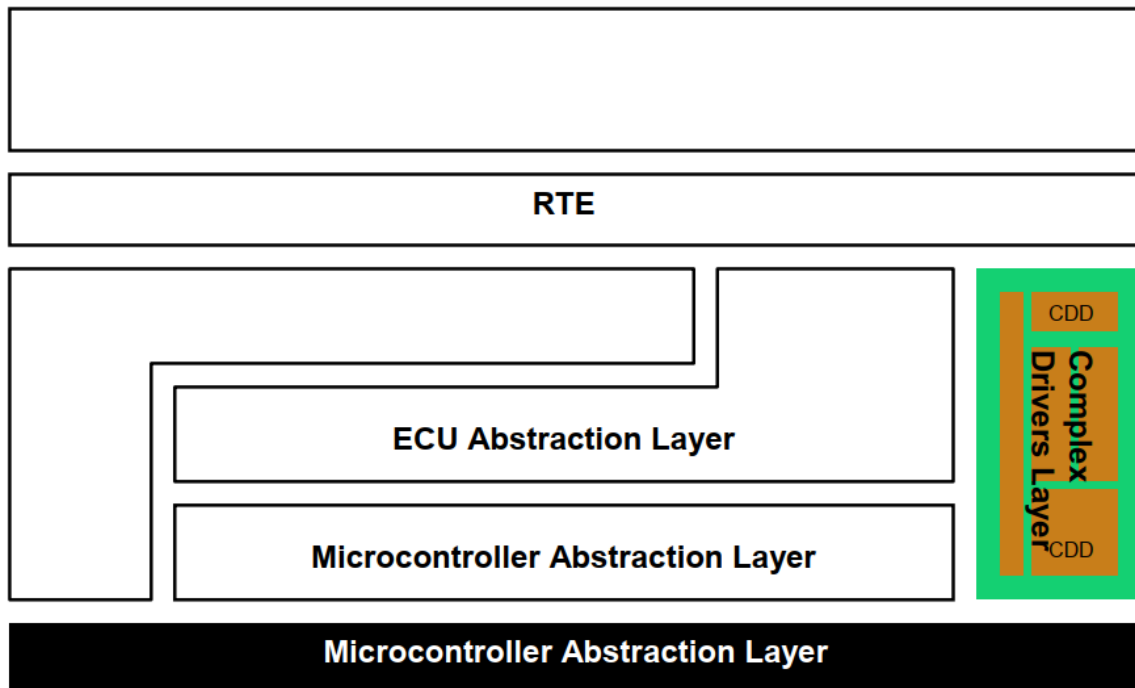
The design of this module follows the specifications of the complex driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the AUTOSAR_EXP_CDDDesignAndIntegrationGuideline.pdf.

1.2 Introduction to CDD

A Complex Driver is a software entity not standardized by AUTOSAR that can access or be accessed via AUTOSAR Interfaces and/or Basic Software Modules APIs.

According to the AUTOSAR Layered Software Architecture, a CDD is a specific module located in the Complex Drivers Layer of the Basic Software which interacts with standard BSW modules or Rte.

- A CDD may need to interface to modules of the layered software architecture.
- A module of the layered software architecture may need to interface to a CDD.
- A CDD may need to interface SW-Cs via RTE.



1.3 Hardware Summary

The Direct Memory Access (DMA) controller module as a highly programmable data-transfer engine has the capacity to execute complex data transfer with minimal intervention from a host processor. It can be used to transfer data flexibly. The mainly hardware microarchitecture includes three parts:

- A DMA engine that performs source address and destination address calculations, and data movement operate from source address to destination address.
- Local memory containing Transfer configurations for 2 instances and each of the 32 channels.
- The Direct Memory Access Multiplexer (DMAMUX) routing DMA sources to any of the 2*32 DMA channels.

- DMA0 interrupts are only available on core0 and DMA1 interrupts are only available on core1/2. core1/2 share the same DMA1 interrupts.
- Support DMA local memory Error Correction Code (ECC) function.

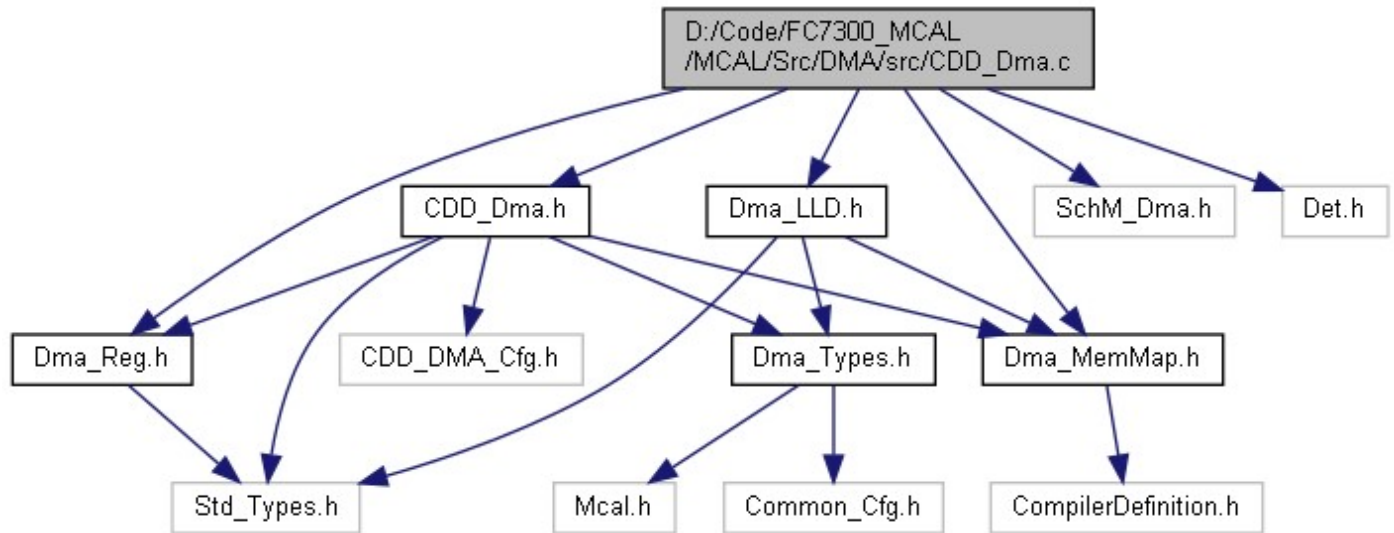
The Direct Memory Access Multiplexer (DMAMUX) routing DMA sources to any of the 32 DMA channels.

Chapter 2 Software Design

2.1 Rejected Requirements

N/A

2.2 File Structure



2.3 Macros

2.3.1 Macros in CDD_Dma.c

- `#define DMA_VENDOR_ID_C 174`
- `#define DMA_AR_RELEASE_MAJOR_VERSION_C 4`
- `#define DMA_AR_RELEASE_MINOR_VERSION_C 6`
- `#define DMA_AR_RELEASE_REVISION_VERSION_C 0`
- `#define DMA_SW_MAJOR_VERSION_C 0`
- `#define DMA_SW_MINOR_VERSION_C 4`
- `#define DMA_SW_PATCH_VERSION_C 0`

2.3.2 Macros in CDD_Dma.h

- `#define DMA_VENDOR_ID 174`
- `#define DMA_AR_RELEASE_MAJOR_VERSION 4`

- **#define DMA_AR_RELEASE_MINOR_VERSION 6**
- **#define DMA_AR_RELEASE_REVISION_VERSION 0**
- **#define DMA_SW_MAJOR_VERSION 0**
- **#define DMA_SW_MINOR_VERSION 4**
- **#define DMA_SW_PATCH_VERSION 0**
- **#define DMA_INDEX(COREID) ((uint8)Dma_Config.pDmaCoreConfig[COREID]->DmaInstanceNumber)**
InstanceId of this Dma driver.
- **#define DMA_E_INVALID_CHANNEL_U8 ((uint8)0x0B)**
API service used with a channel out of range.
- **#define DMA_E_UNINIT_U8 ((uint8)0x0C)**
API service used without module initialization.
- **#define DMA_E_ALREADY_INITIALIZED_U8 ((uint8)0x0D)**
API DMA_Init service called when the DMA driver and the Hardware are already initialized.
- **#define DMA_E_PARAM_VINFO_U8 ((uint8)0x0F)**
API DMA_GetVersionInfo is called and the parameter version info is invalid (e.g. NULL).
- **#define DMA_E_PARAM_CONFIG_U8 ((uint8)0x12U)**
API DMA_Init service called with wrong parameter.
- **#define DMA_E_INIT_FAILED_U8 ((uint8)0x13U)**
API DMA_Init service called with wrong parameter.
- **#define DMA_E_PARAM_CONFIG_INVALID_ADDRESS_U8 ((uint8)0x14U)**
API Dma_Init service called with wrong parameter.
- **#define DMA_E_PARAM_CONFIG_INVALID_SIZE_U8 ((uint8)0x15U)**
API Dma_Init service called with wrong parameter.
- **#define DMA_E_DEINIT_FAILED_U8 ((uint8)0x16U)**
API Dma_DeInit service execution failed.
- **#define DMA_GETVERSIONINFO_ID_U8 ((uint8)0x17U)**
API service ID for DMA_GetVersionInfo function.

- **#define DMA_E_INVALID_REGINDEX_U8** ((uint8)0x18U)
API DMA_ConfigInput service called with wrong parameter register Index.
- **#define DMA_E_INVALID_DMA_SEL_U8** ((uint8)0x19U)
API DMA_ConfigInput service called with wrong parameter selection.
- **#define DMA_E_INVALID_DMA_TRIGGER_U8** ((uint8)0x1AU)
API DMA_ConfigInput service called with wrong parameter trigger.
- **#define DMA_E_DMA_LOCKED_U8** ((uint8)0x1BU)
API DMA_ConfigInput service called when register is locked.
- **#define DMA_INIT_ID_U8** ((uint8)0x23U)
API service ID for DMA_Init function.
- **#define DMA_CONFIG_ID_U8** ((uint8)0x54U)
API service ID for DMA_ConfigInput function.
- **#define DMA_CONFIG_LOCK_U8** ((uint8)0x55U)
API service ID for DMA_TrgMuxConfigLock function
- **#define DMA_DEINIT_ID_U8** ((uint8)0x56U)
API service ID for DMA_DeInit function.
- **#define DMA_START_CH_ID_U8** ((uint8)0x57U)
API service ID for DMA_StartChannel function.
- **#define DMA_SET_INNER_CH_ID_U8** ((uint8)0x58U)
Service ID for DMA_SetInnerLinkChannel function.
- **#define DMA_SET_OUTER_CH_ID_U8** ((uint8)0x59U)
API service ID for DMA_SetOuterLinkChannel function.
- **#define DMA_SET_PRIORITY_ID_U8** ((uint8)0x5AU)
API service ID for DMA_SetChannelPriority function.
- **#define DMA_SET_INNER_ATTR_ID_U8** ((uint8)0x5BU)
API service ID for DMA_SetChannelLoopOffsetAndNBYTES function.
- **#define DMA_TRANSF_COMPLETE_ID_U** ((uint8)0x5CU)
API service ID for Dma_CheckIfTransferCompleted function.
- **#define DMA_TRANSF_ACTIVE_ID_U8** ((uint8)0x5DU)
API service ID for Dma_CheckIfTransferActive function.

- **#define DMA_SET_SADDR_ID_U8** ((uint8)0x5EU)
API service ID for Dma_SetCfgSaddr function.
- **#define DMA_SET_SOFF_ID_U8** ((uint8)0x5FU)
API service ID for Dma_SetCfgSoff function.
- **#define DMA_SET_DLAST_ID_U8** ((uint8)0x60U)
API service ID for Dma_SetCfgDlast function.
- **#define DMA_SET_DOFF_ID_U8** ((uint8)0x61U)
API service ID for Dma_SetCfgDoff function.
- **#define DMA_SET_SLAST_ID_U8** ((uint8)0x62U)
API service ID for Dma_SetCfgSlast function.
- **#define DMA_SET_MOD_SIZE_ID_U8** ((uint8)0x63U)
API service ID for Dma_SetCfgSModuloAndSize function.
- **#define DMA_EN_HW_REQ_ID_U8** ((uint8)0x64U)
API service ID for Dma_EnableHWRequest function.
- **#define DMA_DIS_HW_REQ_ID_U8** ((uint8)0x65U)
API service ID for Dma_DisableHWRequest function.
- **#define DMA_SET_DONE_INT_ID_U8** ((uint8)0x66U)
API service ID for Dma_SetCfgCompleteInterrupt function.
- **#define DMA_CLR_DONE_FLAG_ID_U8** ((uint8)0x67U)
API service ID for Dma_ClearDoneFlag function.
- **#define DMA_GET_BEGIN_COUNTER_ID_U8** ((uint8)0x68U)
API service ID for Dma_GetBeginOuterLoopCounter function.
- **#define DMA_GET_CURRENT_COUNTER_ID_U8** ((uint8)0x69U)
API service ID for Dma_GetCurrentOuterLoopCounter function.
- **#define DMA_SET_OUTER_COUNTER_ID_U8** ((uint8)0x6AU)
API service ID for Dma_UpdateOuterLoopCounter function.
- **#define DMA_CAL_SRC_LAST_OFFSET_ID_U8** ((uint8)0x6BU)
API service ID for Dma_CalculateSrcChannelLastOffset function.

- **#define DMA_CAL_DES_LAST_OFFSET_ID_U8** ((uint8)0x6CU)
API service ID for Dma_CalculateDesChannelLastOffset function.
- **#define DMA_CAL_UPDATE_CONFIGSET_ID_U8** ((uint8)0x6DU)
API service ID for DMA_UpdateChannelConfigSet function.
- **#define DMA_CANCEL_TRANSFER_ID_U8** ((uint8)0x6EU)
API service ID for Dma_CancelTransfer function.
- **#define DMA_ERR_CANCEL_TRANSFER_ID_U8** ((uint8)0x6FU)
API service ID for Dma_ErrorCancelTransfer function.
- **#define DMA_HALT_ID_U8** ((uint8)0x70U)
API service ID for Dma_Halt function.
- **#define DMA_RESUME_ID_U8** ((uint8)0x71U)
API service ID for Dma_Resume function.

2.3.3 Macros in Dma_Types.h

- **#define DMA_TYPES_VENDOR_ID** 174
- **#define DMA_TYPES_MODULE_ID** 27
- **#define DMA_TYPES_AR_RELEASE_MAJOR_VERSION** 4
- **#define DMA_TYPES_AR_RELEASE_MINOR_VERSION** 6
- **#define DMA_TYPES_AR_RELEASE_REVISION_VERSION** 0
- **#define DMA_TYPES_SW_MAJOR_VERSION** 0
- **#define DMA_TYPES_SW_MINOR_VERSION** 4
- **#define DMA_TYPES_SW_PATCH_VERSION** 0

2.4 Enums

2.4.1 Enumerations in Dma_Types.h

2.4.1.1 Dma_InstanceType

Enumeration	Dma_InstanceType	
Description	Defines the available DMA Instances.	
Values	Value	Description
	DMA_INSTANCE_0 = 0U	DMA instance _0.

	DMA_INSTANCE_1 = 1U	DMA instance _1.
	DMA_INSTANCE_MAX	= DMA total instance numbers.
	DMA_INSTANCE_COUNT	

2.4.1.2 Dma_RunningStatusType

Enumeration	Dma_RunningStatusType	
Description	Defines the size of data in one transfer.	
Values	Value	Description
	DMA_RUNNING_STATUS_IDLE = 0	The DMA engine is idle.
	DMA_RUNNING_STATUS_ACTIVE = 1	The DMA engine is transferring data.

2.4.1.3 Dma_ArbitrationAlgorithmType

Enumeration	Dma_ArbitrationAlgorithmType	
Description	DMA channel arbitration algorithm used in the channel arbitration phase.	
Values	Value	Description
	DMA_ARBITRATION_ALGORITHM_FIXED_PRIORITY = 0U	Use the fixed priority for arbitration.
	DMA_ARBITRATION_ALGORITHM_ROUND_ROBIN = 1U	Use the channel number for arbitration, higher channel number has a higher priority.

2.4.1.4 Dma_DriverStateType;

Enumeration	Dma_DriverStateType;	
Description	Driver states list. Details Enumeration listing the different states of the driver.	
Values	Value	Description
	DMA_NOT_INITIALIZED = 0U	Dma has not been initialized.
	DMA_INITIALIZED = 1U	Dma has been initialized.
	DMA_INITIAL_ERROR = 2U	Dma encountered an error during initialization.

2.4.1.5 Dma_TranSizeType

Enumeration	Dma_TranSizeType	
Description	Defines the size of data in one transfer. One transfer can contain multiple blocks, and one block may contain multiple data, this parameter specifies the size of data that the DMA engine will access one time in the memory.	
Values	Value	Description
	DMA_TRAN_SIZE_1B = 0	The size of data in one transfer is 1 byte
	DMA_TRAN_SIZE_2B = 1	The size of data in one transfer is 2 bytes
	DMA_TRAN_SIZE_4B = 2	The size of data in one transfer is 4 bytes
	DMA_TRAN_SIZE_8B = 3	The size of data in one transfer is 8 bytes
	DMA_TRAN_SIZE_32B = 5	The size of data in one transfer is 32 bytes

2.4.1.6 Dma_IncrementModeType

Enumeration	Dma_TranSizeType	
Description	Defines the size of data in one transfer. One transfer can contain multiple blocks, and one block may contain multiple data, this parameter specifies the size of data that the DMA engine will access one time in the memory.	
Values	Value	Description
	DMA_INCREMENT_DISABLE= 0	The data address not increase.
	DMA_INCREMENT_DATA_SIZE = 1	The data address increase by the data size.
	DMA_INCREMENT_DATA_SIZE_4BYTE_ALIGNED = 2	The data address increase by the data size, and is 4 byte aligned.

2.4.1.7 DmaMux_RequestSourceType

Enumeration	DmaMux_RequestSourceType	
Description	DMA Mux hardware trigger source list.	
Values	Value	Description
	DMA_REQ_DISABLED = 0U	No hardware request
	DMA_REQ_FCIIC0_RX = 1U	FCIIC0_RX
	DMA_REQ_FCIIC0_TX = 2U	FCIIC0_TX
	DMA_REQ_FCIIC1_RX = 3U	FCIIC1_RX
	DMA_REQ_FCIIC1_TX = 4U	FCIIC1_TX
	DMA_REQ_FCSPI0_RX = 5U	FCSPI0_RX
	DMA_REQ_FCSPI0_TX = 6U	FCSPI0_TX
	DMA_REQ_FCSPI1_RX = 7U	FCSPI1_RX
	DMA_REQ_FCSPI1_TX = 8U	FCSPI1_TX
	DMA_REQ_FCSPI2_RX = 9U	FCSPI2_RX
	DMA_REQ_FCSPI2_TX = 10U	FCSPI2_TX
	DMA_REQ_FCSPI3_RX = 11U	FCSPI3_RX
	DMA_REQ_FCSPI3_TX = 12U	FCSPI3_TX
	DMA_REQ_FCSPI4_RX = 13U	FCSPI4_RX
	DMA_REQ_FCSPI4_TX = 14U	FCSPI4_TX
	DMA_REQ_FCSPI5_RX = 15U	FCSPI5_RX
	DMA_REQ_FCSPI5_TX = 16U	FCSPI5_TX
	DMA_REQ_FCSPI6_RX = 17U	FCSPI6_RX
	DMA_REQ_FCSPI6_TX = 18U	FCSPI6_TX
	DMA_REQ_FCSPI7_RX = 19U	FCSPI7_RX
	DMA_REQ_FCSPI7_TX = 20U	FCSPI7_TX
	DMA_REQ_OSPI_RX = 21U	OSPI_RX
	DMA_REQ_OSPI_TX = 22U	OSPI_TX
	DMA_REQ_FLEXCAN0 = 23U	FLEXCAN0
	DMA_REQ_FLEXCAN1 = 24U	FLEXCAN1
	DMA_REQ_FLEXCAN2 = 25U	FLEXCAN2
	DMA_REQ_FLEXCAN3 = 26U	FLEXCAN3

DMA_REQ_FLEXCAN4 = 27U	FLEXCAN4
DMA_REQ_FLEXCAN5 = 28U	FLEXCAN5
DMA_REQ_FLEXCAN6 = 29U	FLEXCAN6
DMA_REQ_FLEXCAN7 = 30U	FLEXCAN7
DMA_REQ_FLEXCAN8 = 31U	FLEXCAN8
DMA_REQ_FLEXCAN9 = 32U	FLEXCAN9
DMA_REQ_ADC0 = 33U	ADC0
DMA_REQ_ADC1 = 34U	ADC1
DMA_REQ_ADC2 = 35U	ADC2
DMA_REQ_ADC3 = 36U	ADC3
DMA_REQ_SDDF_CH0 = 37U	SDDF_CH0
DMA_REQ_SDDF_CH1 = 38U	SDDF_CH1
DMA_REQ_SDDF_CH2 = 39U	SDDF_CH2
DMA_REQ_SDDF_CH3 = 40U	SDDF_CH3
DMA_REQ_FCUART0_RX = 41U	FCUART0_RX
DMA_REQ_FCUART0_TX = 42U	FCUART0_TX
DMA_REQ_FCUART1_RX = 43U	FCUART1_RX
DMA_REQ_FCUART1_TX = 44U	FCUART1_TX
DMA_REQ_FCUART2_RX = 45U	FCUART2_RX
DMA_REQ_FCUART2_TX = 46U	FCUART2_TX
DMA_REQ_FCUART3_RX = 47U	FCUART3_RX
DMA_REQ_FCUART3_TX = 48U	FCUART3_TX
DMA_REQ_FCUART4_RX = 49U	FCUART4_RX
DMA_REQ_FCUART4_TX = 50U	FCUART4_TX
DMA_REQ_FCUART5_RX = 51U	FCUART5_RX
DMA_REQ_FCUART5_TX = 52U	FCUART5_TX
DMA_REQ_FCUART6_RX = 53U	FLEXCAN1
DMA_REQ_FCUART6_TX = 54U	FLEXCAN2
DMA_REQ_FCUART7_RX = 55U	FCUART7_RX
DMA_REQ_FCUART7_TX = 56U	FCUART7_TX
DMA_REQ_FCUART8_RX = 57U	FCUART8_RX
DMA_REQ_FCUART8_TX = 58U	FCUART8_TX
DMA_REQ_FCUART9_RX = 59U	FCUART9_RX
DMA_REQ_FCUART9_TX = 60U	FCUART9_TX
DMA_REQ_FCUART10_RX = 61U	FCUART10_RX
DMA_REQ_FCUART10_TX = 62U	FCUART10_TX
DMA_REQ_FCUART11_RX = 63U	FCUART11_RX
DMA_REQ_FCUART11_TX = 64U	FCUART11_TX
DMA_REQ_FCUART12_RX = 65U	FCUART12_RX
DMA_REQ_FCUART12_TX = 66U	FCUART12_TX
DMA_REQ_FCUART13_RX = 67U	FCUART13_RX
DMA_REQ_FCUART13_TX = 68U	FCUART13_TX
DMA_REQ_FCUART14_RX = 69U	FCUART14_RX

DMA_REQ_FCUART14_TX = 70U	FCUART14_TX
DMA_REQ_FCUART15_RX = 71U	FCUART15_RX
DMA_REQ_FCUART15_TX = 72U	FCUART15_TX
DMA_REQ_FCUART16_RX = 73U	FCUART16_RX
DMA_REQ_FCUART16_TX = 74U	FCUART16_TX
DMA_REQ_FCUART17_RX = 75U	FCUART17_RX
DMA_REQ_FCUART17_TX = 76U	FCUART17_TX
DMA_REQ_PORTA = 77U	PORTA
DMA_REQ_PORTB = 78U	PORTB
DMA_REQ_PORTC = 79U	PORTC
DMA_REQ_PORTD = 80U	PORTD
DMA_REQ_PORTE = 81U	PORTE
DMA_REQ_PORTF = 82U	PORTF
DMA_REQ_PORTG = 83U	PORTG
DMA_REQ_PORTH = 84U	PORTH
DMA_REQ_PORTI = 85U	PORTI
DMA_REQ_AONTIMER0 = 86U	AONTIMER0
DMA_REQ_CMP0 = 87U	CMP0
DMA_REQ_CMP1 = 88U	CMP1
DMA_REQ_CMP2 = 89U	CMP2
DMA_REQ_PTIMER0 = 90U	PTIMER0
DMA_REQ_PTIMER1 = 91U	PTIMER1
DMA_REQ_PTIMER2 = 92U	PTIMER2
DMA_REQ_PTIMER3 = 93U	PTIMER3
DMA_REQ_ENET_PPS_TIMER_CH0 = 94U	ENET_PPS_TIMER_CH0
DMA_REQ_ENET_PPS_TIMER_CH1 = 95U	ENET_PPS_TIMER_CH1
DMA_REQ_ENET_PPS_TIMER_CH2 = 96U	ENET_PPS_TIMER_CH2
DMA_REQ_ENET_PPS_TIMER_CH3 = 97U	ENET_PPS_TIMER_CH3
DMA_REQ_FTU0_ALL_CH_OR = 98U	FTU0_ALL_CH_OR
DMA_REQ_FTU1_ALL_CH_OR = 99U	FTU1_ALL_CH_OR
DMA_REQ_FTU2_ALL_CH_OR = 100U	FTU2_ALL_CH_OR
DMA_REQ_FTU3_ALL_CH_OR = 101U	FTU3_ALL_CH_OR
DMA_REQ_FTU4_ALL_CH_OR = 102U	FTU4_ALL_CH_OR
DMA_REQ_FTU5_ALL_CH_OR = 103U	FTU5_ALL_CH_OR
DMA_REQ_FTU6_ALL_CH_OR = 104U	FTU6_ALL_CH_OR
DMA_REQ_FTU7_ALL_CH_OR = 105U	FTU7_ALL_CH_OR
DMA_REQ_FTU8_ALL_CH_OR = 106U	FTU8_ALL_CH_OR
DMA_REQ_FTU9_ALL_CH_OR = 107U	FTU9_ALL_CH_OR
DMA_REQ_FTU10_ALL_CH_OR = 108U	FTU10_ALL_CH_OR
DMA_REQ_FTU11_ALL_CH_OR = 109U	FTU11_ALL_CH_OR
DMA_REQ_SENT0_CH0_FAST = 110U	SENT0_CH0_FAST
DMA_REQ_SENT0_CH1_FAST = 111U	SENT0_CH1_FAST
DMA_REQ_SENT0_CH2_FAST = 112U	SENT0_CH2_FAST

DMA_REQ_SENT0_CH3_FAST= 113U	SENT0_CH3_FAST
DMA_REQ_SENT1_CH0_FAST= 114U	SENT1_CH0_FAST
DMA_REQ_SENT1_CH1_FAST= 115U	SENT1_CH1_FAST
DMA_REQ_SENT1_CH2_FAST= 116U	SENT1_CH2_FAST
DMA_REQ_SENT1_CH3_FAST= 117U	SENT1_CH3_FAST
DMA_REQ_SENT0_CH0_SLOW =118U	SENT0_CH0_SLOW
DMA_REQ_SENT0_CH1_SLOW = 119U	SENT0_CH1_SLOW
DMA_REQ_SENT0_CH2_SLOW = 120U	SENT0_CH2_SLOW
DMA_REQ_SENT0_CH3_SLOW = 121U	SENT0_CH3_SLOW
DMA_REQ_SENT1_CH0_SLOW = 122U	SENT1_CH0_SLOW
DMA_REQ_SENT1_CH1_SLOW = 123U	SENT1_CH1_SLOW
DMA_REQ_SENT1_CH2_SLOW = 124U	SENT1_CH2_SLOW
DMA_REQ_SENT1_CH3_SLOW = 125U	SENT1_CH3_SLOW
DMA_REQ_ALWAYS_ENABLE_0 = 126U	ALWAYS_ENABLE_0
DMA_REQ_ALWAYS_ENABLE_1 = 127U	ALWAYS_ENABLE_1

2.5 Typedef

2.5.1 Typedefs in Dma_Types.h

- typedef void (*Dma_TransferCompleteCallbackType)(void)

This type shall be chosen in order to implement a transfer complete callback function prototype.

- typedef void (*Dma_TransferErrorCallbackType)(void)

This type shall be chosen in order to implement a transfer complete callback function prototype.

2.6 Structures

2.6.1 Dma_ChannelConfigType

Structure	Dma_ChannelConfigType
Description	Used to configure the DMAMUX and DMA channel register.
Diagram	N/A
Data Fields	- uint8 u8PartitionId Specify which CPU this channel belongs to. - uint8 u8Dmachannelid Dma channel number being configured. - uint8 u8DmaChannelPriority DMA channel priority. It is only valid in fixed arbitration mode. - Boolean bDmaMuxPeriodModeEnable Enables the periodic trigger capability for the triggered DMA channel, just channel 0 to channel 3 supported. - Boolean bDmaDoneInterruptEnable Enable the DMA channel complete Interrupt. - boolean bDmaHalfDoneInterruptEnable Enable the DMA channel half complete Interrupt.

	• boolean bDmaErrorInterruptEnable Enable the DMA channel error Interrupt. • DmaMux_RequestSourceType eDmaMuxRequestSource Select the hardware DMA request source. • Dma_TransferCompleteCallbackType pDmaDoneISRCallBack Pointer to callback notification for DMA channel complete interrupt. • Dma_TransferErrorCallbackType pDmaErrorISRCallBack Pointer to callback notification for DMA channel error interrupt. • Uint8 u8DmaISRRouter Interrupt route to core selection. • boolean bInnerChannelSelfChain Dma innerloop channel chain enable.
--	---

2.6.2 Dma_ChannelType

Structure	Dma_ChannelType
Description	Contains the basic info for DMA channel, should be filled by user.
Diagram	N/A
Data Fields	• uint32 u32DmaSADDR Memory address pointing to the source data. • uint32 u32DmaDADDR Memory address pointing to the destination data. • boolean bDmaAutoStopEnable Enable the DMA request automatically when outer loop is completed. • boolean bSrcCircularBufferEn Source circular buffer enable. • uint32 u32SrcCircBufferSize Source circular buffer size. • boolean bDestCircularBufferEn Destination circular buffer enable. • uint32 u32DestCircBufferSize Destination circular buffer size. • DMA_TransferSizeType eDmaSSIZE Source data transfer size. • DMA_TransferSizeType eDmaDSIZE Destination data transfer size. • Dma_IncrementModeType eDmaSrcIncrementMode Source data increment size after DMA engine transferred a datum. • Dma_IncrementModeType eDmaDesIncrementMode Destination data increment size after DMA engine transferred a datum. • uint32 s32DmaSLAST Last source address adjustment. • uint32 s32DmaDLAST Destination last address adjustment.

	• uint32 u32DmaNBYTES Inner byte total transfer count. • uint16 u16DmaOuterLoopCounter the number of innerloops in one transfer. • uint32 s32DmaLOFF A sign-extended offset applied to the source or destination address to form the next-state value after the inner loop completes. • boolean bDmaSILOE Source innerloop offset enable. • boolean bDmaDILOE Destination innerloop Offset enable.
--	--

2.6.3 Dma_CoreSpecificChConfigType

Structure	Dma_CoreSpecificChConfigType
Description	Contains the information related to the hardware setup for DMA and DMAMux.
Diagram	N/A
Data Fields	• uint8 DmaInstanceNumber DMA Instance number. • uint8 DmaNumberOfChConfiguredPerCore Number of DMA channel configured • const Dma_ChannelConfigType *pDma_pChConfig Pointer to channel config list

2.6.4 Dma_CoreSpecificHwUnitConfigType

Structure	Dma_CoreSpecificChConfigType
Description	Contains the information related to the hardware setup for DMA and DMAMux.
Diagram	N/A
Data Fields	• boolean bDmaEnableInnerLoopMap Enable DMA Inner loop mapping. • boolean bDmaEnableContinuousLinkMode Enable continuous trig mode. • boolean bDmaEnableHaltOnError If enable, any error causes the HALT bit to set. • boolean bDmaEnableRoundRobinMode If enable, round robin arbitration is used for channel selection. • boolean bDmaEnableDebugStopMode If enable, the DMA stalls the start of a new channel when in debug mode. • boolean bDmaEnableMonitorChecker If enable, The DMA enables Monitor Checker. • uint8 bDmaMastercore The corresponding master core of the Dma instance.

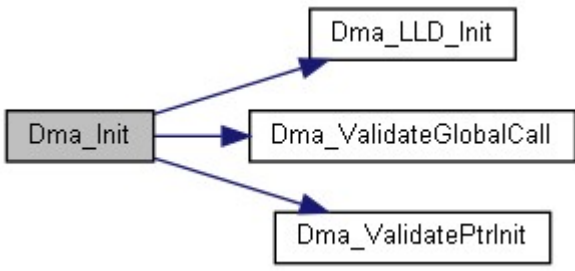
2.6.5 Dma_ConfigType

Structure	Dma_ConfigType
Description	Contains the information related to the hardware setup for DMA and DMAMux.
Diagram	N/A
Data Fields	- const Dma_CoreSpecificChConfigType* pDmaCoreConfig[MCAL_CORES_NUM] Contains the information related to the core specific channel config. - const Dma_CoreSpecificHwUnitConfigType* pDmaHwunitConfig Contains the information related to the core specific hardware unit config. - boolean DmaPartitionsUsed[MCAL_CORES_NUM] Dma used partitions.

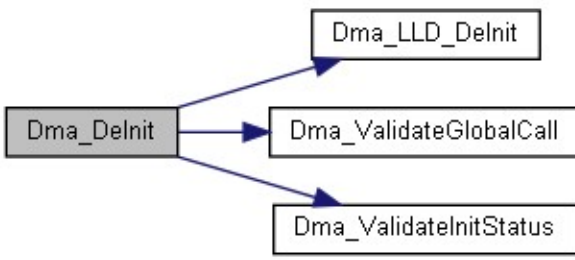
2.7 API Functions

2.7.1 Functions in Cdd_Dma.c

2.7.1.1 void Dma_Init (const Dma_ConfigType *pConfigPtr)

Function	void Dma_Init(const Dma_ConfigType *pConfigPtr)	
Description	Function that initializes the DMA module. This function sets the DMACR register with the value of "pConfigPtr" and set the channel's priority, ECP and DPA.	
Diagram	 <pre> graph LR Dma_Init[Dma_Init] --> Dma_LLD_Init[Dma_LLD_Init] Dma_Init --> Dma_ValidateGlobalCall[Dma_ValidateGlobalCall] Dma_Init --> Dma_ValidatePtrInit[Dma_ValidatePtrInit] </pre>	
Parameters	Parameter	Description
	pConfigPtr	Pointer to configuration set
Returns	void	

2.7.1.2 void Dma_DeInit(void)

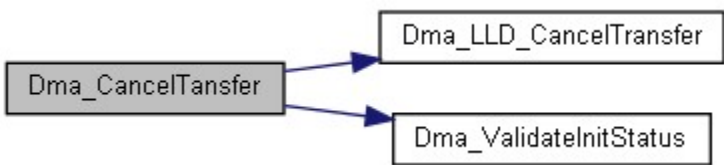
Function	void Dma_DeInit(void)	
Description	This function de-initializes the DMA driver. This service is a non-reentrant function. Returns all underlying hardware to a state comparable to their power on reset state, and de-initialize the DMA driver.	
Diagram	 <pre> graph LR Dma_DeInit[Dma_DeInit] --> Dma_LLD_DeInit[Dma_LLD_DeInit] Dma_DeInit --> Dma_ValidateGlobalCall[Dma_ValidateGlobalCall] Dma_DeInit --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	void	

Returns	void
----------------	------

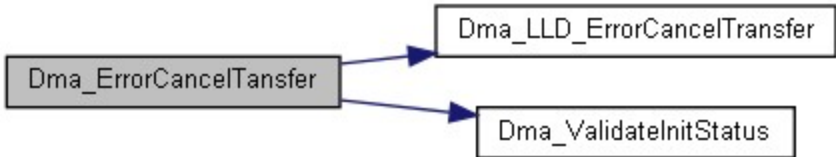
2.7.1.3 void Dma_GetVersionInfo(Std_VersionInfoType * pVersioninfo)

Function	void Dma_GetVersionInfo(Std_VersionInfoType * pVersioninfo)	
Description	This service returns the version information of this module. This service is Non reentrant and returns the version information of this module. The version information includes: - Module Id - Vendor Id - Vendor specific version numbers. If source code for caller and callee of this function is available this function should be realized as a macro. The macro should be defined in the modules header file.	
Diagram	N/A	
Parameters	Parameter	Description
	pVersioninfo	Pointer to where to store the version information of this module.
Returns	N/A	

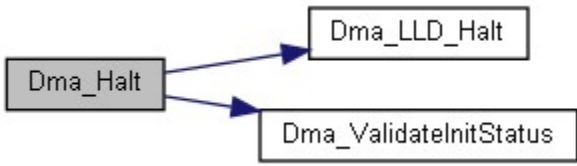
2.7.1.4 Std_ReturnType Dma_CancelTransfer(const Dma_InstanceType eDma_Instance)

Function	Std_ReturnType Dma_CancelTransfer(const Dma_InstanceType eDma_Instance)	
Description	This function is used for cancelling the DMA operation.	
Diagram	 <pre> graph LR Dma_CancelTransfer --> Dma_LLD_CancelTransfer Dma_CancelTransfer --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	eDma_Instance	Dma instance to be operated
Returns	E_OK: cancel operation is successful. E_NOT_OK: cancel operation is failed.	

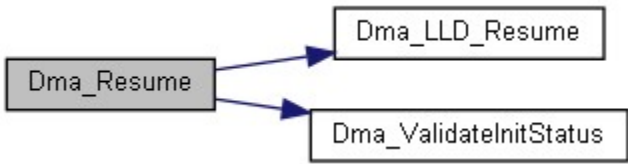
2.7.1.5 Std_ReturnType Dma_ErrorCancelTransfer(const Dma_InstanceType eDma_Instance)

Function	Std_ReturnType Dma_ErrorCancelTransfer(const Dma_InstanceType eDma_Instance)	
Description	This function is used for cancelling the DMA operation with an error flag.	
Diagram	 <pre> graph LR Dma_ErrorCancelTransfer --> Dma_LLD_ErrorCancelTransfer Dma_ErrorCancelTransfer --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	eDma_Instance	Dma instance to be operated
Returns	E_OK: cancel operation is successful E_NOT_OK: cancel operation is failed	

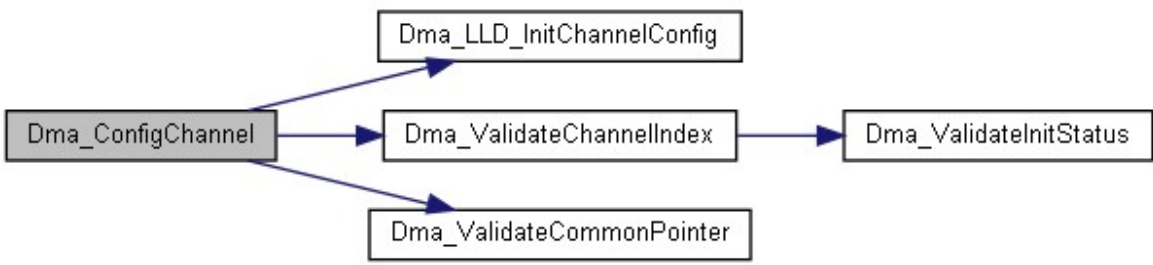
2.7.1.6 void Dma_Halt(const Dma_InstanceType eDma_Instance)

Function	void Dma_Halt(const Dma_InstanceType eDma_Instance)	
Description	This function is used to halt the DMA operations.	
Diagram	 <pre> graph LR Dma_Halt[Dma_Halt] --> Dma_LLD_Halt[Dma_LLD_Halt] Dma_Halt --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	eDma_Instance	Dma instance to be operated
Returns	N/A	

2.7.1.7 void Dma_Resume(const Dma_InstanceType eDma_Instance)

Function	void Dma_Resume(const Dma_InstanceType eDma_Instance)	
Description	This function is used to resume the DMA operations.	
Diagram	 <pre> graph LR Dma_Resume[Dma_Resume] --> Dma_LLD_Resume[Dma_LLD_Resume] Dma_Resume --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	eDma_Instance	Dma instance to be operated
Returns	N/A	

2.7.1.8 Std_ReturnType Dma_ConfigChannel(const uint8 u8Dma_Channel, const Dma_ChannelType* pChannelConfig)

Function	Std_ReturnType Dma_ConfigChannel(const uint8 u8Dma_Channel, const Dma_ChannelType* pChannelConfig)	
Description	This function configures a DMA Channel. This function is reentrant and configures the specified DMA channel	
Diagram	 <pre> graph LR Dma_ConfigChannel[Dma_ConfigChannel] --> Dma_LLD_InitChannelConfig[Dma_LLD_InitChannelConfig] Dma_ConfigChannel --> Dma_ValidateChannelIndex[Dma_ValidateChannelIndex] Dma_ConfigChannel --> Dma_ValidateCommonPointer[Dma_ValidateCommonPointer] Dma_ValidateChannelIndex --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel
	pChannelConfig	Pointer to the channel's descriptor attributes
Returns	E_OK: channel config operation is successful E_NOT_OK: channel config operation is failed	

2.7.1.9 void Dma_SetCfgCompleteInterrupt(uint8 u8Dma_Channel, Boolean bEnable)

Function	void Dma_SetCfgCompleteInterrupt(uint8 u8Dma_Channel, Boolean bEnable)
Description	This function enables the interrupts when outer loop count completes.

Diagram	<pre> graph LR A[Dma_SetCfgCompleteInterrupt] --> B[Dma_LLD_SetCompleteInt] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	bEnable	True: Enable the Interrupt. False: Disable the interrupt
Returns	N/A	

2.7.1.10 void Dma_StartChannel(const uint8 u8Dma_Channel)

Function	void Dma_StartChannel(const Dma_InstanceType eDma_Instance, const uint8 u8Dma_Channel)	
Description	This function starts the specified DMA Channel with SW trigger. This function is reentrant and starts the specified DMA Channel.	
Diagram	<pre> graph LR A[Dma_StartChannel] --> B[Dma_LLD_StartChannel] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	N/A	

2.7.1.11 void Dma_SetInnerLinkChannel(const uint8 u8Dma_Channel, const uint8 u8Dma_TriggerChannel)

Function	void Dma_SetInnerLinkChannel(const uint8 u8Dma_Channel, const uint8 u8Dma_TriggerChannel)	
Description	This function is reentrant and configures the specified Inner linked DMA channel. It only works normally when u16DmaOuterLoopCounter is > 1.	
Diagram	<pre> graph LR A[Dma_SetInnerLinkChannel] --> B[Dma_LLD_SetInnerLinkChannel] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u8Dma_TriggerChannel	Numeric identifier of the next DMA channel.
Returns	N/A	

2.7.1.12 void Dma_SetOuterLinkChannel (const uint8 u8Dma_Channel, const uint8 u8Dma_TriggerChannel)

Function	void Dma_SetOuterLinkChannel (const uint8 u8Dma_Channel, const uint8 u8Dma_TriggerChannel)	
Description	This function configures Outer linked DMA Channel, the linked channel is triggered after the outer counter exhaust. This function is reentrant and configures the specified Inner linked DMA channel.	
Diagram	<pre> graph LR A[Dma_SetOuterLinkChannel] --> B[Dma_LLD_SetOuterLinkChannel] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	

Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u8Dma_TriggerChannel	Numeric identifier of the next DMA channel.
Returns	N/A	

2.7.1.13 void Dma_SetChannelPriority (const uint8 u8Dma_Channel, const uint8 u8Dma_Priority)

Function	void Dma_SetChannelPriority (const uint8 u8Dma_Channel, const uint8 u8Dma_Priority)	
Description	This function sets the priority for the specified DMA Channel. This function is reentrant and sets the priority for the specified DMA Channel.	
Diagram	<pre> graph LR Dma_SetChannelPriority --> Dma_LLD_SetChannelPriority Dma_SetChannelPriority --> Dma_ValidateChannelIndex Dma_ValidateChannelIndex --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u8Dma_Priority	Value for the priority.
Returns	N/A	

2.7.1.14 boolean Dma_SetChannelLoopOffsetAndNBYTES (const uint8 u8Dma_Channel, const DMA_NBYTESIOLType *PConfig)

Function	boolean Dma_SetChannelLoopOffsetAndNBYTES (const uint8 u8Dma_Channel, const DMA_NBYTESIOLType *PConfig)	
Description	This function sets the Inner loop offset and counter for the specified DMA Channel. This function is reentrant and sets the priority for the specified DMA Channel.	
Diagram	<pre> graph LR Dma_SetChannelLoopOffsetAndNBYTES --> Dma_LLD_SetChannelLoopOffsetAndNBYTES Dma_SetChannelLoopOffsetAndNBYTES --> Dma_ValidateChannelIndex Dma_SetChannelLoopOffsetAndNBYTES --> Dma_ValidateCommonPointer Dma_ValidateChannelIndex --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	PConfig	Pointer to the channel's Inner loop attributes.
Returns	True: the inner loop mapping is enabled, the setting is applied.	
	False: the inner loop mapping is disabled, the setting is ignored.	

2.7.1.15 boolean Dma_CheckIfTransferCompleted (uint8 u8Dma_Channel)

Function	boolean Dma_CheckIfTransferCompleted (uint8 u8Dma_Channel)	
Description	This function checks if the DMA transfer is completed. This function is reentrant and checks if the DMA transfer is completed.	

Diagram	<pre>graph LR; A[Dma_CheckIfTransferCompleted] --> B[Dma_LLD_CheckIfCompleteFlag]; A --> C[Dma_ValidateChannelIndex]; C --> D[Dma_ValidateInitStatus];</pre>					
Parameters	<table><tr><th>Parameter</th><th>Description</th></tr><tr><td>u8Dma_Channel</td><td>Numeric identifier of the DMA channel.</td></tr></table>	Parameter	Description	u8Dma_Channel	Numeric identifier of the DMA channel.	
Parameter	Description					
u8Dma_Channel	Numeric identifier of the DMA channel.					
Returns	True: DMA transfer has completed. False: DMA transfer is in progress.					

2.7.1.16 boolean Dma_CheckIfTransferActive (uint8 u8Dma_Channel)

Function	boolean Dma_CheckIfTransferActive (uint8 u8Dma_Channel)					
Description	This function checks if a DMA transfer is active. This function is reentrant and checks if a DMA transfer is active.					
Diagram	<pre>graph LR; A[Dma_CheckIfTransferActive] --> B[Dma_LLD_CheckIfActiveFlag]; A --> C[Dma_ValidateChannelIndex]; C --> D[Dma_ValidateInitStatus];</pre>					
Parameters	<table><tr><th>Parameter</th><th>Description</th></tr><tr><td>u8Dma_Channel</td><td>Numeric identifier of the DMA channel.</td></tr></table>	Parameter	Description	u8Dma_Channel	Numeric identifier of the DMA channel.	
Parameter	Description					
u8Dma_Channel	Numeric identifier of the DMA channel.					
Returns	True: DMA transfer is active. False: DMA transfer is not active.					

2.7.1.17 void Dma_SetCfgSlast (uint8 u8Dma_Channel, sint32 s32Slast)

Function	void Dma_SetCfgSlast (uint8 u8Dma_Channel, sint32 s32Slast)	
Description	Set the address adjustment applied to the source address after outer loop finished.	
Diagram	<pre>graph LR; A[Dma_SetCfgSlast] --> B[Dma_LLD_SetSLAST]; A --> C[Dma_ValidateChannelIndex]; C --> D[Dma_ValidateInitStatus];</pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	s32Slast	Adjustment value added to the source address at the completion of the major iteration count.
Returns	N/A	

2.7.1.18 void Dma_SetCfgSaddr (uint8 u8Dma_Channel, uint32 u32Saddr)

Function	void Dma_SetCfgSaddr (uint8 u8Dma_Channel, uint32 u32Saddr)		
Description	Set the source address of the DMA channel.		
Diagram	<pre>graph LR; A[Dma_SetCfgSaddr] --> B[Dma_LLD_SetSADDR]; A --> C[Dma_ValidateChannelIndex]; C --> D[Dma_ValidateInitStatus];</pre>		
Parameters	Parameter	Description	

	u8Dma_Channel	Numeric identifier of the DMA channel.
	u32Saddr	Address to set in SADDR.
Returns	N/A	

2.7.1.19 void Dma_SetCfgSoff (uint8 u8Dma_Channel, sint16 s16Soff)

Function	void Dma_SetCfgSoff (uint8 u8Dma_Channel, sint16 s16Soff)	
Description	Set the source data offset of the DMA channel.	
Diagram	<pre> graph LR Dma_SetCfgSoff --> Dma_LLD_SetSOFF Dma_SetCfgSoff --> Dma_ValidateChannelIndex Dma_ValidateChannelIndex --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	s16Soff	Source address offset.
Returns	N/A	

2.7.1.20 void Dma_SetCfgDlast (uint8 u8Dma_Channel, sint32 s32Dlast)

Function	void Dma_SetCfgDlast (uint8 u8Dma_Channel, sint32 s32Dlast)	
Description	Set the address adjustment applied to the destination address after outer loop finished.	
Diagram	<pre> graph LR Dma_SetCfgDlast --> Dma_LLD_SetDLAST Dma_SetCfgDlast --> Dma_ValidateChannelIndex Dma_ValidateChannelIndex --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	s32Dlast	Adjustment value added to the destination address at the completion of the major iteration count.
Returns	N/A	

2.7.1.21 void Dma_SetCfgDaddr (uint8 u8Dma_Channel, uint32 u32Daddr)

Function	void Dma_SetCfgDaddr (uint8 u8Dma_Channel, uint32 u32Daddr)	
Description	Set the destination address of the DMA channel.	
Diagram	<pre> graph LR Dma_SetCfgDaddr --> Dma_LLD_SetDADDR Dma_SetCfgDaddr --> Dma_ValidateChannelIndex Dma_ValidateChannelIndex --> Dma_ValidateInitStatus </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u32Saddr	Address to set in DADDR
Returns	N/A	

2.7.1.22 void Dma_SetCfgDoff (uint8 u8Dma_Channel, sint16 s16Doff)

Function	void Dma_SetCfgDoff (uint8 u8Dma_Channel, sint16 s16Doff)	
Description	Set the destination data offset of the DMA channel.	
Diagram	<pre> graph LR Dma_SetCfgDoff[Dma_SetCfgDoff] --> Dma_LLD_SetDOFF[Dma_LLD_SetDOFF] Dma_SetCfgDoff --> Dma_ValidateChannelIndex[Dma_ValidateChannelIndex] Dma_ValidateChannelIndex --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u32Daddr	Destination address offset.
Returns	N/A	

2.7.1.23 void Dma_SetCfgSModuloAndSize (uint8 u8Dma_Channel, uint8 u8SModulo, DMA_TranSizeType eSSize)

Function	void Dma_SetCfgSModuloAndSize (uint8 u8Dma_Channel, uint8 u8SModulo, DMA_TranSizeType eSSize)	
Description	Set the source address aligned modulo and source data size of the DMA channel.	
Diagram	<pre> graph LR Dma_SetCfgSModuloAndSize[Dma_SetCfgSModuloAndSize] --> Dma_LLD_SetSModuloAndSize[Dma_LLD_SetSModuloAndSize] Dma_SetCfgSModuloAndSize --> Dma_ValidateChannelIndex[Dma_ValidateChannelIndex] Dma_ValidateChannelIndex --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u8SModulo	Source Address Modulo.
	eSSize	Source data transfer size.
Returns	N/A	

2.7.1.24 void Dma_SetCfgDModuloAndSize (uint8 u8Dma_Channel, uint8 u8DModulo, DMA_TranSizeType eDSize)

Function	void Dma_SetCfgDModuloAndSize (uint8 u8Dma_Channel, uint8 u8DModulo, DMA_TranSizeType eDSize)	
Description	Set the destination address aligned modulo and destination data size of the DMA channel.	
Diagram	<pre> graph LR Dma_SetCfgDModuloAndSize[Dma_SetCfgDModuloAndSize] --> Dma_LLD_SetDModuloAndSize[Dma_LLD_SetDModuloAndSize] Dma_SetCfgDModuloAndSize --> Dma_ValidateChannelIndex[Dma_ValidateChannelIndex] Dma_ValidateChannelIndex --> Dma_ValidateInitStatus[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u8DModulo	Destination Address Modulo.
	eDSize	Destination data transfer size.
Returns	N/A	

2.7.1.25 void Dma_EnableHwRequest (uint8 u8Dma_Channel)

Function	void Dma_EnableHwRequest (uint8 Dma_Channel)
Description	This function is used for enabling the hardware request for a given DMA channel.

Diagram	<pre> graph LR A[Dma_EnableHwRequest] --> B[Dma_LLD_EnableHwRequest] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	N/A	

2.7.1.26 void Dma_DisableHwRequest (uint8 u8Dma_Channel)

Function	void Dma_DisableHwRequest (uint8 u8Dma_Channel)	
Description	This function is used for disabling the hardware request for a given DMA channel.	
Diagram	<pre> graph LR A[Dma_DisableHwRequest] --> B[Dma_LLD_DisableHwRequest] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	N/A	

2.7.1.27 void Dma_ClearDoneFlag(uint8 u8Dma_Channel)

Function	void Dma_ClearDoneFlag(uint8 u8Dma_Channel)	
Description	This function is used for clearing the Channel Done flag.	
Diagram	<pre> graph LR A[Dma_ClearDoneFlag] --> B[Dma_LLD_ClearDoneFlag] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	N/A	

2.7.1.28 uint16 Dma_GetBeginOuterLoopCounter(uint8 u8Dma_Channel)

Function	uint16 Dma_GetBeginOuterLoopCounter(uint8 u8Dma_Channel)	
Description	This function is used for getting the begin outer counter for a specified channel.	
Diagram	<pre> graph LR A[Dma_GetBeginOuterLoopCounter] --> B[Dma_LLD_GetBeginOuterLoopCounter] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	Beginning outerloop counter	

2.7.1.29 uint16 Dma_GetCurrentOuterLoopCounter(uint8 u8Dma_Channel)

Function	uint16 Dma_GetCurrentOuterLoopCounter(uint8 u8Dma_Channel)	
-----------------	--	--

Description	This function is used for getting the current outer counter for a specified channel.	
Diagram	<pre> graph LR A[Dma_GetCurrentOuterLoopCounter] --> B[Dma_LLD_GetCurrentOuterLoopCounter] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
Returns	Current outerloop counter	

2.7.1.30 void Dma_UpdateOuterLoopCounter(uint8 u8Dma_Channel, uint16 u16Counter)

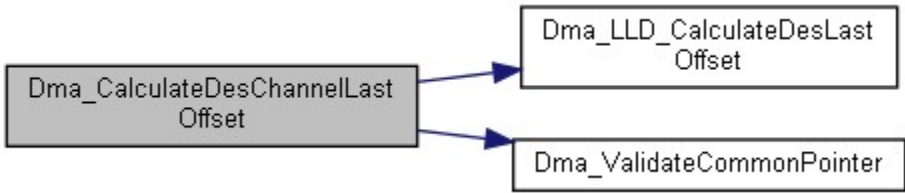
Function	void Dma_UpdateOuterLoopCounter(uint8 u8Dma_Channel, uint16 u16Counter)	
Description	This function is used for updating the outer count bits.	
Diagram	<pre> graph LR A[Dma_UpdateOuterLoopCounter] --> B[Dma_LLD_UpdateOuterLoopCounter] A --> C[Dma_ValidateChannelIndex] C --> D[Dma_ValidateInitStatus] </pre>	
Parameters	Parameter	Description
	u8Dma_Channel	Numeric identifier of the DMA channel.
	u16Counter	Outerloop counter number
Returns	N/A	

2.7.1.31 Std_ReturnType Dma_CalculateSrcChannelLastOffset(const Dma_ChannelType* PConfig, sint32 *s32SrcLastOffset)

Function	Std_ReturnType Dma_CalculateSrcChannelLastOffset(const Dma_ChannelType* PConfig, sint32 *s32SrcLastOffset)	
Description	This function calculates the last offset of source, and the last offset value will be returned through input parameters s32SrcLastOffset. This function is only used to calculate the last offset to loop back to origin start address when transfer finished.	
Diagram	<pre> graph LR A[Dma_CalculateSrcChannelLastOffset] --> B[Dma_LLD_CalculateSrcLastOffset] A --> C[Dma_ValidateCommonPointer] </pre>	
Parameters	Parameter	Description
	PConfig	Pointer to the channel's Inner loop attributes.
	s32SrcLastOffset	Pointer to the address used to store the SrcLastOffset.
Returns	E_OK: calculate is finished. E_NOT_OK: calculate is canceled.	

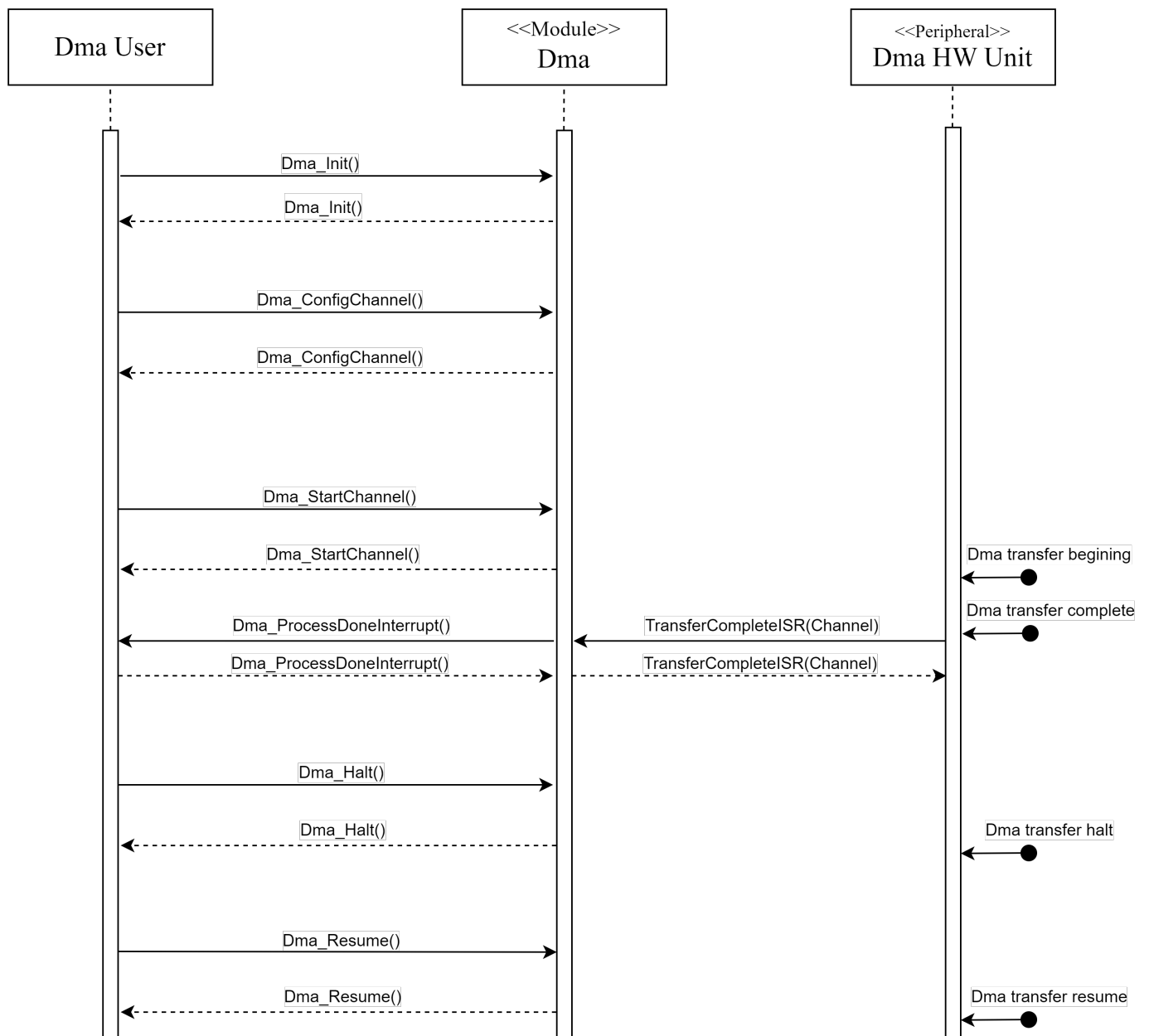
2.7.1.32 Std_ReturnType Dma_CalculateDesChannelLastOffset(const Dma_ChannelType* PConfig, sint32 *s32DestLastOffset)

Function	Std_ReturnType Dma_CalculateDesChannelLastOffset(const Dma_ChannelType* PConfig, sint32 *s32DestLastOffset)	
-----------------	---	--

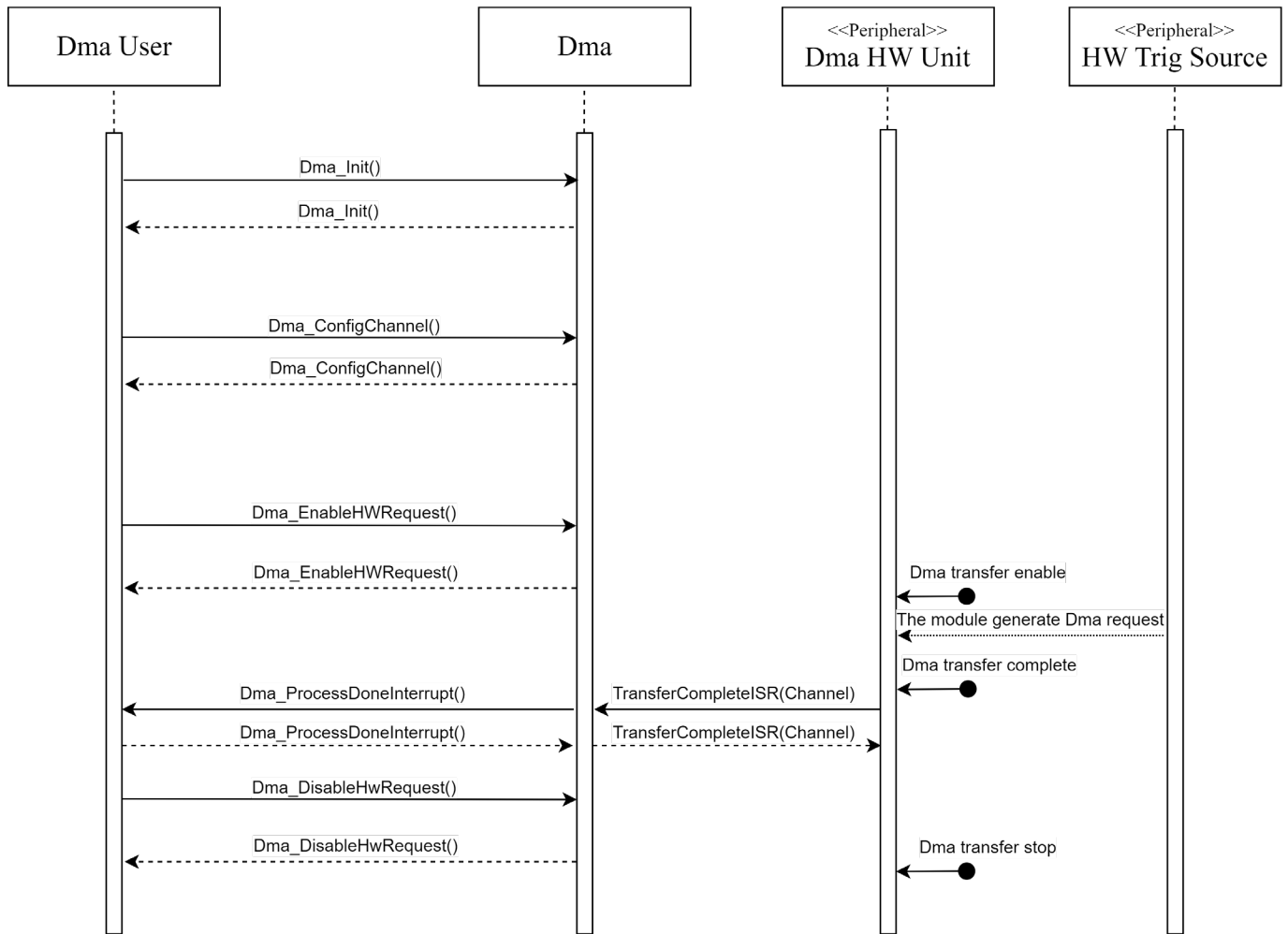
Description	This function calculates the last offset of destination, and the last offset value will be returned through input parameters s32DestLastOffset. This function is only used to calculate the last offset to loop back to origin start address when transfer finished.	
Diagram	 <pre> graph LR A[Dma_CalculateDesChannelLast Offset] --> B[Dma_LLD_CalculateDesLast Offset] A --> C[Dma_ValidateCommonPointer] </pre>	
Parameters	Parameter	Description
	PConfig	Pointer to the channel's Inner loop attributes.
	s32DestLastOffset	Pointer to the address used to store the DesLastOffset.
Returns	E_OK: calculate is finished. E_NOT_OK: calculate is canceled.	

2.8 WAPI Sequence Diagram

2.8.1 DMA Initialization and Requested by Software



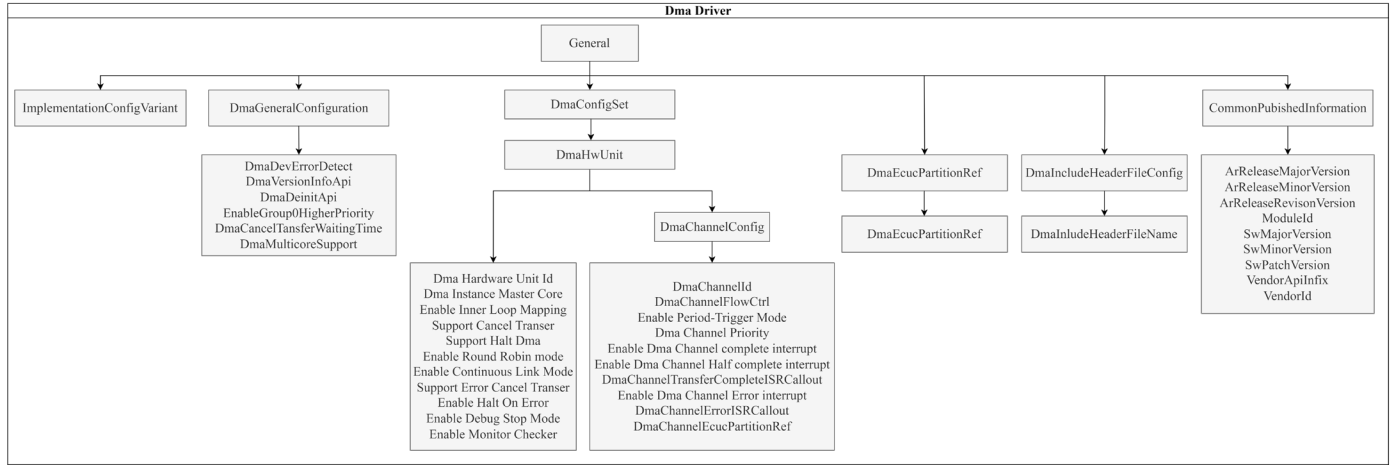
2.8.2 DMA Initialization and Requested by Hardware



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The container inclusion relation is shown below:



3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Config Variant
	Default	VariantPreCompile

3.2.2 DMAGeneral

Container	DMAGeneral	
Description	Vendor specific: Configuration of general DMA parameters.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.2.1 DmaDevErrorDetect

Container	DMADevErrorDetect	
Description	Switches the Development Error Detection and Notification on or off. - true: Enabled.	

	- false: Disabled.	
Screenshot	Dma Dev Error Detect  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Dma Dev Error Detect
	Default	false

3.2.2.2 Dma_VersionInfoApi

Container	Dma_VersionInfoApi	
Description	Vendor specific: Enables/Disables the get version info API function.	
Screenshot	Dma VersionInfo Api  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Dma Version Info Api
	Default	false

3.2.2.3 Dma_DeInitApi

Container	Dma_DeInitApi	
Description	Vendor specific: Enables/Disables the DMA_DeInit API.	
Screenshot	Dma DeInit Api  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Dma DeInit Api
	Default	false

3.2.2.4 EnableGRP1HighPriority

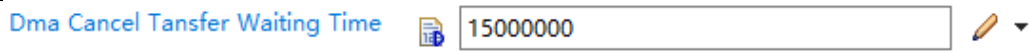
Container	EnableGRP0HighPriority	
Description	Vendor specific: Enable: Group1 has higher priority than group0.Disable: Group0 has higher priority than group1.	
Screenshot	Enable Group1 Higher Priority  	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Group1 Higher Priority
	Default	false

3.2.2.5 Dma_CancelTransWaitTime

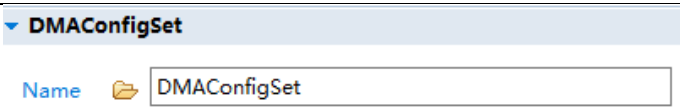
Container	Dma_CancelTransWaitTime	
Description	Dma cancel transfer waiting time, timed out will exit waiting. Unit: Clock cycle	
Screenshot	Dma Cancel Tansfer Waiting Time  15000000 	

Properties	Property	Value
	Type	INTEGER
	Label	Dma Cancel Transfer Waiting Time
	Default	15000000
Requirements	ECUC_DMA_006	

3.2.2.6 Dma Multicore Support

Container	DmaMulticoreSupport	
Description	Dma cancel transfer waiting time, timed out will exit waiting. Unit: Clock cycle	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Dma Cancel Transfer Waiting Time
	Default	15000000
Requirements	ECUC_DMA_006	

3.2.3 DMAConfigSet

Container	DMAConfigSet	
Description	This container is the base for a multiple configuration set.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.3.1 Dma Hardware Unit Id

Container	DmaHwUnitId	
Description	Numeric ID of the HW Unit.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Dma Hardware Unit Id
	Default	0->1

3.2.3.2 Dma Instance Master Core

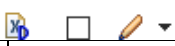
Container	Dma Instance Master Core	
Description	Specify the relevant master core of the instance.	
Screenshot		

Properties	Property	Value
	Type	INTEGER
	Label	Dma Hardware Unit Id
	Default	0->1

3.2.3.3 EnableInnerLoopMapping

Container	EnableInnerLoopMapping	
Description	- Enable: DMA_CFG_NBYTES_MLOFFYESn is redefined as the fields including individual enable, an offset, and the NBYTES field. The individual enable fields allow the inner loop offset to be applied to the source address, the destination address, or both. The NBYTES field is reduced when either offset is enabled. - Disable: DMA_CFG_NBYTES_MLNO is defined as a 32-bit NBYTES field.	
Screenshot	Enable Inner Loop Mapping 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Inner Loop Mapping
	Default	True

3.2.3.4 Enable Continuous Link Mode

Container	EnableContinuousLinkMode	
Description	- Enable: A minor loop channel link made to itself goes through channel arbitration before being activated again. - Disable: A minor loop channel link made to itself does not go through channel arbitration before being activated again. Upon minor loop completion, the channel activates again if that channel has a minor loop channel link enabled and the link channel is itself. This effectively applies the minor loop offsets and restarts the next minor loop.	
Screenshot	Enable Continuous Link Mode 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Continuous Link Mode
	Default	false

3.2.3.5 Support Cancel Transer

Container	SupportCancelTranser	
Description	Support the API to cancel the remaining data transfer.	
Screenshot	Support Cancel Transer 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Support Cancel Transfer
	Default	false

3.2.3.6 Support ErrorCancel Transter

Container	SupportErrorCancelTranster	
Description	Support the API to cancel the remaining data transfer and report a related Error in ES register.	
Screenshot	Support Error Cancel Transter    	
Properties	Property	Value
	Type	BOOLEAN
	Label	Support Error Cancel Transfer
	Default	false

3.2.3.7 Support Halt Dma

Container	SupportHaltDma	
Description	Support the API to stall the start of any new channels and executing channels are allowed to complete.	
Screenshot	Support Halt DMA    	
Properties	Property	Value
	Type	BOOLEAN
	Label	Support Halt DMA
	Default	false

3.2.3.8 Enable Halt On Error

Container	EnableHaltOnError	
Description	- Enable: Once any error reports on ES register, the DMA module would halt itself and all service requests are ignored until the HALT bit is cleared. - Disable: Normal operation.	
Screenshot	Enable Halt On Error    	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Halt On Error
	Default	false

3.2.3.9 Enable Round Robin Mode

Container	EnableRoundRobinMode	
Description	- Enable: Round robin arbitration is used for channel selection. - Disable: Fixed priority arbitration is used for channel selection.	
Screenshot	Enable Round Robin mode    	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Round Robin mode
	Default	false

3.2.3.10 Enable Debug Stop Mode

Container	EnableDebugStopMode	
------------------	---------------------	--

Description	- Enable: The DMA stalls the start of a new channel when in debug mode. - Disable: The DMA continues to operate when in debug mode.	
Screenshot	Enable Debug Stop Mode 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Debug Stop Mode
	Default	false

3.2.3.11 Enable Monitor Checker

Container	EnableMonitorChecker	
Description	Vendor specific:Enable: The DMA enables Monitor Checker.Disable: The DMA disables Monitor Checker.	
Screenshot	Enable Monitor Checker 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Monitor Checker
	Default	false

3.2.4 DmaChannelConfig

Container	DmaChannelConfig	
Description	This container contains the configuration parameters for the DMA channels.	
Screenshot		
Properties	Property	Value
	Type	MAP
	Label	N/A
	Default	N/A

3.2.4.1 DmaChannelId

Container	DmaChannelId	
Description	DMA channel number being configured.	
Screenshot	DmaChannelId (dynamic range) 	
Properties	Property	Value
	Type	INTEGER
	Label	N/A
	Default	0

3.2.4.2 DmaChannelFlowCtrl

Container	DmaChannelFlowCtrl	
Description	Assign the flow controller of this channel transfer. - DMA_REQ_DISABLED: DMA is the flow controller. For example, DMA START trigger, inner link trigger and outer link trigger - Others: HW Peripheral trigger selected as source is the flow controller.	

	By default, value is 'DMA_REQ_DISABLED' (No HW flow control).	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	N/A
	Default	DMA_REQ_DISABLED

3.2.4.3 EnablePeriodTriggerMode

Container	EnablePeriodTriggerMode	
Description	- Enable: the DMAMUX is in Periodic Trigger mode. - Disable: the DMAMUX is in Periodic Normal mode. Note: Only channel0-3 support this mode.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Period-Trigger Mode
	Default	false

3.2.4.4 DmaChannelPriority

Container	DmaChannelPriority	
Description	Channel priority level for fixed-priority arbitration. Range: 0 ~ 15.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	DMA Channel Priority
	Default	0

3.2.4.5 EnableChannelCompleetInterrupt

Container	EnableChannelCompleetInterrupt	
Description	- Enable: enable the channel complete interrupt. - Disable: disable the channel complete interrupt.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable DMA Channel complete interrupt
	Default	false

3.2.4.6 EnableChannelHalfCompleetInterrupt

Container	EnableChannelHalfCompleetInterrupt	
Description	- Enable: enable the channel half complete interrupt. - Disable: disable the channel half complete interrupt.	

Screenshot	Enable DMA Channel Half complete interrupt  ☐ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable DMA Channel Half complete interrupt
	Default	false

3.2.4.7 DMAChannelTransferCompleteISRCallout

Container	DMAChannelTransferCompleteISRCallout	
Description	This parameter defines the existence and the name of a callout function that is called by driver ISR on a successful DMA transfer of a block. If this parameter is omitted no callout shall take place.	
Screenshot	DMAChannelTransferCompleteISRCallout  ☐ 	
Properties	Property	Value
	Type	FUNCTION
	Label	N/A
	Default	NULL_PTR

3.2.4.8 EnableChannelErrorInterrupt

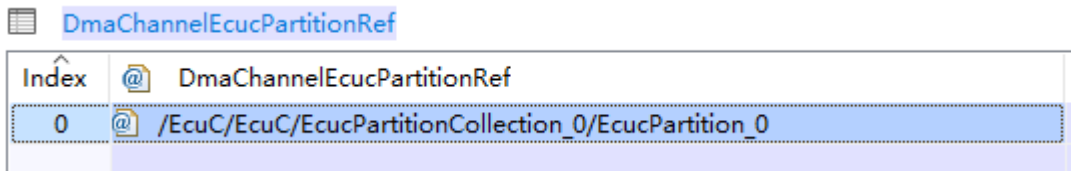
Container	EnableChannelErrorInterrupt	
Description	- Enable: enable the channel error interrupt. - Disable: disable the channel error interrupt.	
Screenshot	Enable DMA Channel Error interrupt  ☐ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable DMA Channel Error interrupt
	Default	false

3.2.4.9 DmaChannelErrorISRCallout

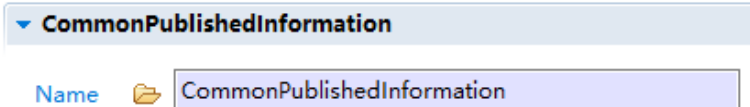
Container	DmaChannelErrorISRCallout	
Description	This parameter defines the existence and the name of a callout function that is called by the driver ISR on a DMA transfer error occurrence. If this parameter is omitted no callout shall take place.	
Screenshot	DmaChannelErrorISRCallout  ☐ 	
Properties	Property	Value
	Type	FUNCTION
	Label	N/A
	Default	NULL_PTR

3.2.4.10 DmaChannelEcucPartitionRef

Container	DmaChannelEcucPartitionRef	
Description	Maps the Dma channel to zero a multiple ECUC partitions. The ECUC partitions referenced are a subset of the ECUC partitions where the Dma driver is mapped to.	

Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Label	N/A
	Default	N/A

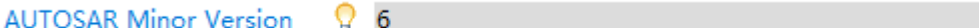
3.2.5 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.5.1 AUTOSAR Major Version

Container	AUTOSAR Major Version	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	4

3.2.5.2 AUTOSAR Minor Version

Container	AUTOSAR Minor Version	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	6

3.2.5.3 AUTOSAR Patch Version

Container	AUTOSAR Patch Version	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	

Screenshot	AUTOSAR Patch Version  0	
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	0

3.2.5.4 Software Major Version

Container	Software Major Version	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Major Version  0	
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	0

3.2.5.5 Software Minor Version

Container	Software Minor Version	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Minor Version  4	
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	4

3.2.5.6 Software Patch Version

Container	Software Patch Version	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Patch Version  0	
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	0

3.2.5.7 Module ID

Container	Module ID	
Description	Module ID of this module from Module List.	
Screenshot	Module ID  27	
Properties	Property	Value

	Type	INTEGER_LABEL
	Label	N/A
	Default	27

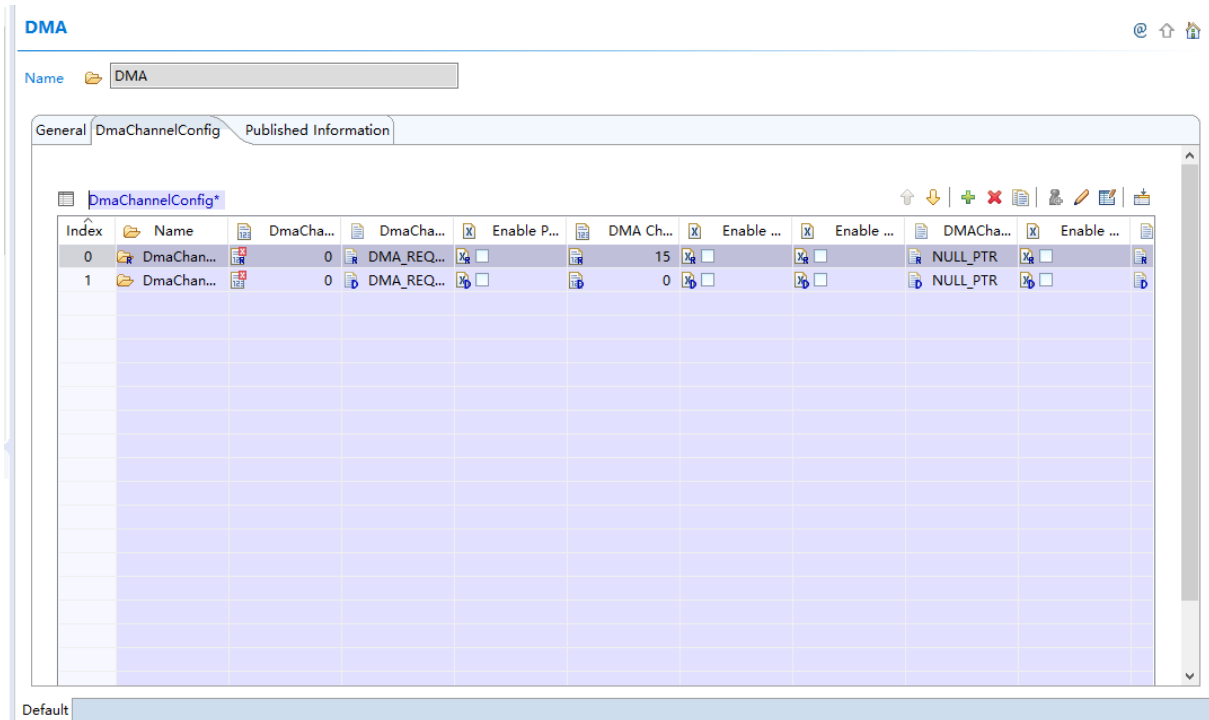
3.2.5.8 Vendor ID

Container	Vendor ID	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	Label	N/A
	Default	174

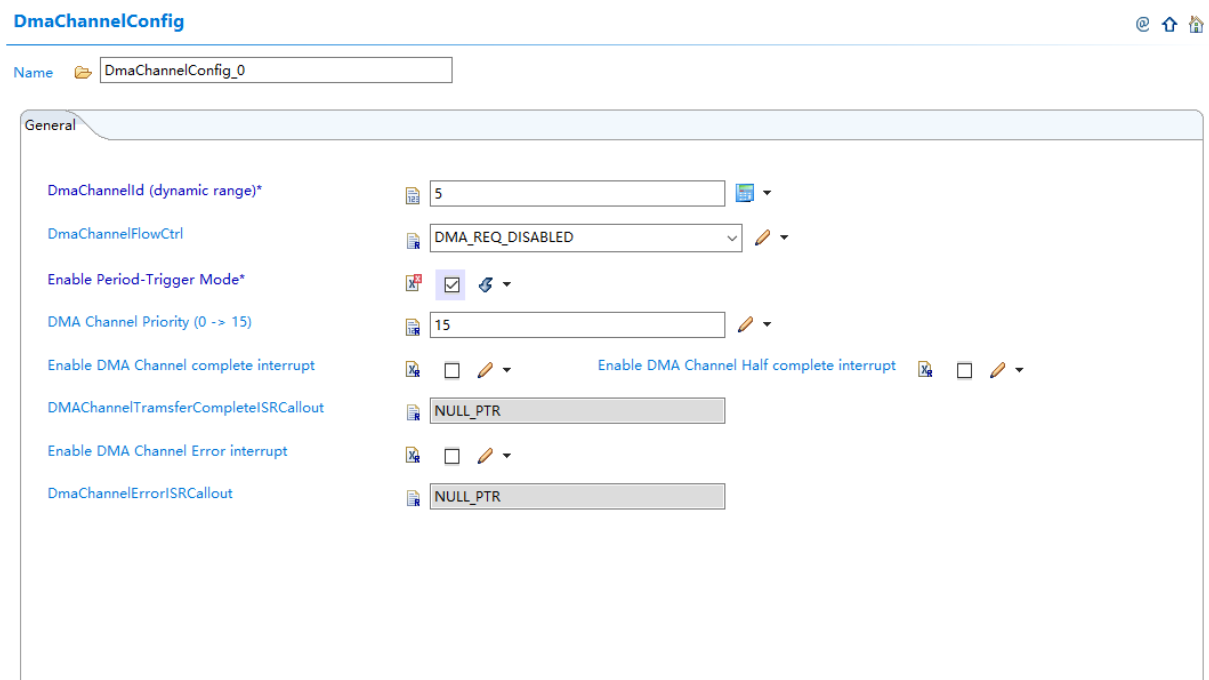
Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

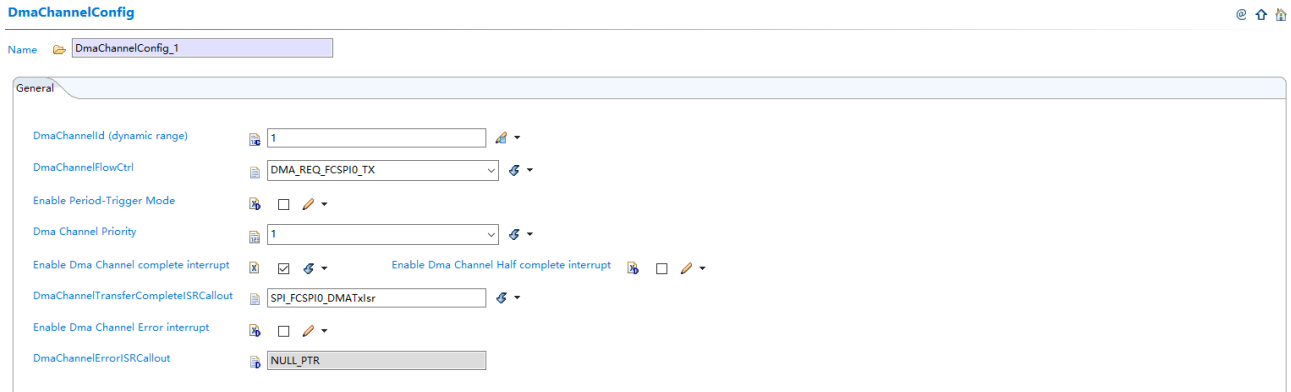
- 1) In the DMA module, each DMA channel only has one configuration setting. That means that duplicate values cannot be used in the DmaChannelId column.



- 2) In DMA channel setting, Period-Trigger Mode only can be used for DMA channels 0-3.



- 3) In DMA channel setting, Dma Channel Priority configuration of all channels must be continuous from 0, cannot repeat and the higher the number, the higher the priority.
- 4) When configuring a DMA channel for software triggering, select "DmaChannelFlowCtrl" as "DMA_REQ_DISABLED".
- 5) When using hardware triggering, it is important to pay attention to the configuration of the callback function interface. Most peripheral modules require this function to ensure the normal operation of other peripheral module state machines and other functions.



- 6) When using hardware triggering, in addition to configuring in the DMA module, it is also necessary to select Enable DMA in the triggering source module.
- 7) When selecting Dma Multicore Support, EcucPartitionRef and DmaChannelEcucPartitionRef shall be configured based on demand.
- 8) Mastercore of instance0 must be core0, and mastercore of instance1 can be of core1 or core2.

4.2 DMA Usage Common Steps

Basically, the DMA module can be configured by following the below 5 steps:

- 1) Configure DMA General.
- 2) According to used DMA channel, add the related DMA channel configuration settings in DmaChannelConfig.
- 3) Choose which API will be used.
- 4) Generate configuration files.
- 5) Implement more detailed channel configuration in the code.

4.3 DMA Configuration Demo

4.3.1 Hardware Trigger

DmaChannelConfig



Name DmaChannelConfig_1

General

DmaChannelId (dynamic range)	1
DmaChannelFlowCtrl	DMA_REQ_FCSPiO_TX
Enable Period-Trigger Mode	☐
Dma Channel Priority	1
Enable Dma Channel complete interrupt	☒ Enable Dma Channel Half complete interrupt ☐
DmaChannelTransferCompleteISRCallout	SPI_FCSPiO_DMATxIsr
Enable Dma Channel Error interrupt	☐
DmaChannelErrorISRCallout	NULL_PTR

4.3.2 Software Trigger

DmaChannelConfig



Name DmaChannelConfig_0

General

DmaChannelId (dynamic range)	0
DmaChannelFlowCtrl	DMA_REQ_DISABLED
Enable Period-Trigger Mode	☐
Dma Channel Priority	0
Enable Dma Channel complete interrupt	☒ Enable Dma Channel Half complete interrupt ☐
DmaChannelTransferCompleteISRCallout	Dma_Channel0_CompleteIsr
Enable Dma Channel Error interrupt	☐
DmaChannelErrorISRCallout	NULL_PTR